universität innsbruck

Bachelor Thesis

Finding optimal solutions to a Teacher Assignment Problem using MILP- and SMT-Solvers

Benedikt Schenk (11831240)
benedikt.schenk@uibk.ac.at

14 October 2022

Supervisor: Manuel Eberl

Eidesstattliche Erklärung

Ich erkläre hiermit an Eides statt durch meine eigenhändige Unterschrift, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Alle Stellen, die wörtlich oder inhaltlich den angegebenen Quellen entnommen wurden, sind als solche kenntlich gemacht.
Ich erkläre mich mit der Archivierung der vorliegenden Bachelorarbeit einverstanden.

14.10.2022
Datum


Unterschrift

Abstract

The area of Teacher Assignment Problems is a topic of high interest, not only for schools and universities. Therefore, over the years, many different solution-strategies evolved, each trying to keep computation time low whilst still finding an optimal or near-optimal solution. We look at a specific real-world version of this problem: mountainbike instructors have to be assigned to courses in geographically different locations according to their distance, preferences, as well as availability constraints. We reformulate the problem as a linear optimization problem over integer variables by linearizing the non-linear parts of the objective function. Two different approaches for the optimization are used and compared: Using the MILP-Solver PuLP and using the SMT-Solver Z3 with the vZ extension. It turns out that PuLP is better suited for the problem.

Contents

1 Introduction

Many sorts of assignment problems exist, and they arise in all sorts of scenarios and fields [9]. The task is to assign agents to tasks and find the best viable combination of assignments. Non-viable are combinations of assignments which do not satisfy given constraints. Assignments are assumed to come with a so-called cost. The optimal solution consists out of the combination of assignments with the lowest possible combined cost. The cost can represent many things, e.g. the distance of a distributor to the customer or the time it takes for workers to complete a job.

The classical assignment problem has n agents and n tasks. The mathematical formulation of the objective is the following:

$$\underset{x_{ij}}{\text{minimize}} \sum_{i=1}^{n}\sum_{j=1}^{n} x_{ij}c_{ij}$$

$$\text{subject to} \quad \sum_{j=1}^{n} x_{ij} = \sum_{j=1}^{n} x_{ji} = 1 \qquad \forall i \in [1, ..., n]$$

$$x_{ij} \in \{0, 1\} \qquad \forall i, j \in [1, ..., n].$$

The cost of assigning agent i to task j is c_{ij} and is given with the problem. As x is not given but needs to be decided, it is a (and in this case the only) decision parameter. x_{ij} is 1 if agent i is assigned to task j, 0 otherwise. The only constraints are that every task must have exactly one agent assigned and every agent must be assigned to exactly one task.

Many other more sophisticated variations of this classical assignment problem exist [18]. One of them is the Teacher Assignment Problem (TAP). Its objective is to assign n teachers (agents) to m courses (tasks) and minimize a cost function. There are many kinds of teacher assignment problems, but the usual kind and the one to be solved in this paper includes the following properties: Teachers are not available for every course, courses may take place simultaneously (meaning that a teacher can teach at most one of those courses), the teachers have preferences for the courses and they each have a desired workload (typically the number of teaching hours they desire to have). Therefore it is a combination of attributes of multiple kinds of assignment problems: the bottleneck assignment problem, the minimum deviation assignment problem, the multi-criteria assignment problem, the assignment problem with side-constraints, and the generalized assignment problem, according to the definitions of D. W. Pentico [18].

It is important to note that all the information of the courses is already given and cannot be changed. This differentiates it from timetabling problems, which also belong

to the education domain. In a timetabling problem, courses need to be assigned to a limited number of timeslots and rooms [9].

Every assignment problem has a matrix associated with it. Usually the rows represent the agents, the columns the tasks and the cells the cost to assign the agent to the task [23]. As assignment problems have been tried to be solved for many years, lots of different solution approaches evolved. Most can be categorized into the following: Exact method, heuristic technique, metaheuristic technique and hybrid [9]. In this paper the exact method of solving with Integer Programming (IP) will be used, for reasons explained in Section 4.1. This is an NP-complete problem [13].

2 Problem description

An actual real-world problem is being solved in this paper. The scenario is the following: A German mountain bike association offers courses in Germany, Austria, Italy and Switzerland. Before the beginning of each year, all courses get scheduled regarding which date they start, how many days they last, where they take place and how many instructors are needed per course. Each course takes place just once. The instructors will be called teachers in the following to achieve consistency with other papers in the field of teacher assignment problems. There is a pool of around 10 – 15 teachers who are all eligible to teach any course. Once a schedule is decided on, it gets sent out to the teachers and each of them must specify the following:

- How many days they would like to work in total
- For which courses they are available
- For the course where they are available: if teaching this course has a low, medium or high priority for them. The high priority is reserved for teachers who created the concept of the course.
- Their place of living (which will be considered as a starting / end point for travels to / from courses)

The task is to find the best combination of teacher assignments, which will be called Assignment Plan (AP) in the following. How good an AP is is given by the cost function, which is based on multiple criteria: the overall distance the teachers need to travel (criterion 1), the priorities the teachers have towards their assigned courses (criterion 2), as well as how well the desired workloads of the teachers were met (criterion 3). Of course, some restrictions must be respected, which are formalized in Section 4.4.

To the best of my knowledge, including criterion 3 is a novelty in the area of TAPs. It can be handled similarly to the more usual priority criterion 2. It is also noteworthy that courses can require several teachers and that they can be at the same time or overlap in time.

Many papers in the area of the TAP assume their defined problem to be satisfiable. This is not the case for the problem at hand, as often there are courses for which not enough teachers marked themselves as available. In practice, teachers then get contacted individually, in the hope that they can be convinced to teach an under-assigned course. This needs to be considered in the solution to this problem.

The program solving this problem will have to give the user power to influence the solution. For one, by letting the user modify parameters, which change how the criteria

are weighted against each other. For another, by letting the user specify a fourth priority-level for teacher–course combinations, which is a hard “teacher x must be assigned for course y” constraint.

3 Motivation

In our real-world problem, every year the assignment plan was created manually by the same person. Let us call this person the Decision Maker (DM). Using his experience, he intuitively assigns teachers to courses until he gets a first viable solution. Then he changes or switches single assignments until eventually the AP seems to not get any better or is good enough.

While it works, it tends to result in dissatisfaction: one, because this person is limited in how much time he can spend on creating the plan and must be satisfied with an imperfect solution, and two, because the assignment plan can be subject to bias. And even if there is none, it still fuels mistrust in the team as teachers feel disadvantaged and not appreciated. In fact, this already led to conflicts in the past and undermined the working morale.

Another reason to automate the creation of the assignment plans is the fact that the association plans to reduce its carbon footprint. The distance traveled by the teachers significantly impacts it and should therefore be minimized. On another note, the teachers need to be paid for each kilometer travelled, so there is also a financial incentive. The expectation is that with automation this distance could be reduced.

Also, the manual creation of assignment plans is time-consuming and therefore costly. Even though few, the criteria and restrictions mentioned in Chapter 2 make finding a good combination by hand very cumbersome. This is due to the sheer number of theoretically possible APs but also as, if given two schedules, you cannot necessarily tell which is the better one on a glance. You need to think about priorities, distances, workloads and having it all balanced.

With a problem size of 30 courses with 2 necessary teachers per course and 10 teachers, who are each available for 10 courses, the DM takes about one full business day to complete the schedule although he has several years of experience and thus has developed a workflow and roughly knows where each teacher lives.

4 Mathematical formulation

4.1 Approach

The fact that the problem of this thesis cannot be solved by a brute force algorithm should become apparent very quickly when looking at the number of theoretically possible APs.

Let C be the number of courses, T the number of teachers and R_c the required number of teachers for a course c $\in \{1, ..., C\}$. The complexity is the following (assuming perfect teacher availability and no courses overlapping in time):

$$O\left(\prod_{c=1}^{C}\binom{T}{R_c}\right).$$

Already with 20 teachers and 40 courses, each course requiring 2 teachers, there are over a googol combinations:

$$(20 \cdot 19)^{40} \approx 1.6 \cdot 10^{103}.$$

Fortunately, as described in Chapter 1, many approaches to this problem exist. In this thesis the problem gets reformulated as an optimization problem, where an objective function needs to be optimized while satisfying certain constraint functions. These constraint functions give a boundary on which assignments are allowed. The objective function in this case is a cost function, which describes how "bad" an assignment is and must therefore be minimized to yield the optimal solution. The actual decision variables are boolean values, as each teacher can either be assigned to teach a course or not. As boolean values are discrete, this would make it an IP-problem [24]. However, some non-discrete helper decision variables will be needed in order to incorporate how much the assigned number of workdays deviates from the desired number of workdays for the different teachers, as to be seen in Section 4.4. Therefore the decision variables are not only discrete, making it a Mixed-Integer Programming (MIP)-problem. [6]

This has two main advantages. For one, you will be guaranteed to find the optimal solution (or one of them if several optimums exist) and do not need to be contented with a near-optimum. For another, this solution is very flexible. If the requirements were to change, the code could easily be modified to account for it. If for example two teachers must not work together, or some teachers can only teach certain courses if they taught another course beforehand, this could quickly be added as additional constraints. Also the concept of which AP is "good" can be changed, by modifying the cost function. There is no need to rethink a whole algorithm.

The issue with this approach is complexity. As mentioned in Chapter 1, IP problems are known to be NP-complete. This means that no algorithm exists, which is guaranteed to find the solution of the problem within polynomial time [12]. For such complex domains, so-called incomplete solvers exist, which work with simplified calculations and partial consistency amongst constraints [13].

However, Non-Linear Optimization (NLO) problems encompass a vast range of problems, as János D. Pintér [19] explains. Therefore in this paper the approach of formulating the problem as a Mixed-Integer Linear Programming (MILP) is chosen in order to reduce complexity and with it, runtime.

The big drawback of this approach is that the problem needs to be formulated as a linear problem, which comes with certain compromises. This is elaborated in the following section 4.3.

4.2 Parameters

The following parameters are used to formulate and solve the problem.

Input parameters:

T	The number of teachers
C	The number of courses
R_c	The required number of teachers for course c; $c = 1, ..., C$
P_{tc}	The priority of teacher t to teach course c. $c = 1, ..., C; t = 1, ..., T$ 0 means teacher could teach the course, 1 means teacher wants to teach the course, 2 means teacher should teach the course, 3 means teacher must teach the course.
W_t	The desired number of workdays of teacher t. $t = 1, ..., T$
L_c	The length (or duration) of course c in days; $c = 1, ..., C$
D_{tc}	The distance between teacher t and course c; c = 1, ..., C; t = 1, ..., T
S	A set of sets, specifying all events that intersect time-wise $S \subset \mathcal{P}(1, ..., C)$

Decision parameters:

x_{tc}	If teacher t is assigned to course c then 1, otherwise 0

Auxiliary parameter:

A	The worst case sum of distances between courses and their assigned teachers.

Auxiliary decision parameters:

δ_t^+	Indicates the absolute overload: by how much the number of assigned workdays exceed teacher t's desired number of workdays.
δ_t^-	Indicates the absolute underload: by how much the number of assigned workdays subceed teacher t's desired number of workdays.
Δ	The maximum over-/underload amongst all teachers, relative to their desired workloads.

Hyper parameters:

α_1 A weighting parameter for the overall distance.

α_2 A weighting parameter for the priorities.

α_3 A weighting parameter for the desired number of workdays.

λ A value between 0 and 1. It balances the optimisation of the average, and the maximum relative deviation of desired to assigned workloads. Further explanation in Section 4.3.

4.3 Cost function

The cost function should, for a given AP, return a value that indicates how "bad" it is. In this paper, a bad assignment plan is defined as one where the distances between teachers and courses are big (criterion 1), the average priority of teachers towards their assigned courses is low (criterion 2) and the average deviation between desired and assigned workload is high and unequal between teachers (criterion 3). Criteria 1 and 2 can be included as two simple linear functions. Criterion 3 could be formalized by a cost function that linearly adds up all relative or absolute deviations per teacher but this would not address equality and could result in single teachers being very discontent. In other words, an AP for two teachers A and B where both have the same medium deviation should be preferred over an assignment plan where A has a low deviation and B a high deviation. According to Domenech & Lusa [8], in such cases usually the standard deviation is minimized. As it is non-linear, they offer another approach: Instead of minimizing the standard deviation, one can minimize both the average relative deviation and the maximum relative deviation of desired to assigned workload. With the proper weights this should achieve a similar effect as using the standard deviation. This weight will be called λ and will be determined later in this section. The bigger λ is, the more importance is given to minimizing the maximum workload deviation of a teacher. The lower, the more importance is given to minimizing the average workload deviation.

The three criteria need to be addressed in the cost function. Yet how the criteria should be weighed to each other cannot be objectively determined, as it is in the eye of the beholder, how important each of them is. Therefore the weighting parameters α_1, α_2 and α_3 will be introduced to let the DM manipulate the cost function in a way that it represents his intentions. The cost function is defined as follows (cf. Domenech & Lusa [8]):

$$\begin{aligned} & \frac{\alpha_1}{A} \cdot \sum_{t=1}^{T} \sum_{c=1}^{C} x_{tc} \cdot D_{tc} \\ & + \alpha_2 \cdot \left(1 - \frac{\sum_{t=1}^{T} \sum_{c=1}^{C} x_{tc} \cdot P_{tc}}{\left(\sum_{c=1}^{C} R_c\right) \cdot \max\limits_{\forall t \forall c} P_{tc}}\right) \\ & + \alpha_3 \cdot \left(\frac{\lambda}{T} \cdot \left(\sum_{t=1}^{T} \frac{\delta_t^+ + \delta_t^-}{W_t}\right) + (1 - \lambda) \cdot \Delta\right). \end{aligned} \tag{4.1}$$

Already included in the formula is the normalization of each of the three criteria in order to make them comparable. Note that the overall distance gets normalized by dividing it with the overall distance in a worst-case-solution. This ensures good normalization but needs a separate computation. The computation, however, is much less complex and therefore should not increase computation time significantly. It can be obtained by the following optimization:

$$A = \underset{x}{\text{maximize}}\Big(\sum_{t=1}^{T} \sum_{c=1}^{C} x_{tc} \cdot D_{tc}\Big). \tag{4.2}$$

The maximization must respect the constraints of Section 4.4.

The normalization of the priority criterion can be done similiar to the normalization of the distance, by dividing by the sum of priorities in a worst-case scenario. For the problem at hand this was assessed to not be necessary. Yet in other scenarios, where the distribution of priorities is very unbalanced, this can be advisable.

As Domenech & Lusa [8] point out, the λ factor should be selected in a way such that the standard deviation gets minimized. This always depends on the concrete scenario that needs to get solved. As each year the initial situation changes, with different courses, teachers, distances etc., there is not just one specific scenario the value should be optimized for but many. As a consequence, the value should be selected to deliver the best average result for a range of scenarios, which should all represent different but plausible scenarios. How this is done can be read in Section 6.1. Worth noting is that the teachers usually have a very different desire in the amount of days they want to work with an average standard deviation with a value of 15.

The average standard deviation of desired to assigned workload was computed for 10 values of λ and 50 randomized scenarios for each. It can be observed, that the λ-value and the calculated standard deviation do not linearly correlate, which is not surprising due to the nonlinear nature of the standard deviation (see Figure 4.1). Furthermore it can be seen that for λ-values bigger than 0.8, the standard deviation rapidly increases to its maximum of 5.38 for $\lambda = 1$. This shows how important it is to include the maximum workload deviation in the cost function. For values smaller or equal to 0.7, the standard deviation stays very constant, with the biggest difference being 0.29 points. The minimum value was obtained for $\lambda = 0.4$ with a standard deviation of 3.33. The λ-value will be inserted into the cost-function 4.1.

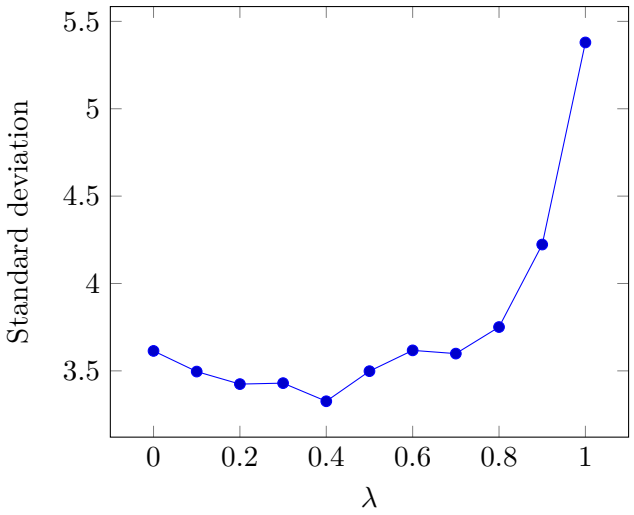


Figure 4.1: The standard deviation of the desired to the assigned workload for different values of λ.

4.4 Constraints

Several constraints need to be respected when minimizing the cost function. There are three rather trivial constraints: Teachers cannot teach courses which overlap in time (see 4.3), teachers who have priority 3 towards a course must be assigned to the course (see 4.4) and the number of teachers assigned to a course must not exceed the number of teachers needed for the course (see 4.5).

$$\sum_{c_i \in s} x_{tc_i} \leq 1 \qquad \forall t \in [1, ..., T] \; \forall s \in S \tag{4.3}$$

$$P_{tc} = 3 \Rightarrow \; x_{tc} = 1 \qquad \forall t \in [1, ..., T] \; \forall c \in [1, ..., C] \tag{4.4}$$

$$\sum_{t=1}^{T} x_{tc} \leq R_c \qquad \forall c \in [1, ..., C] \tag{4.5}$$

Note that only the maximum number of teachers assigned to a course is constrained, not the minimum. This is due to the fact that it is not given that for all courses the needed number of teachers can be found, as stated in Chapter 2. Therefore the constraint is not that every course has the needed number of teachers assigned, but that overall there are as many teacher–course assignments as possible. This is realized by using lexicographic optimization, where first one objective function is optimized and then its result is added as a constraint for another objective function before optimizing it [20].

Let y denote the maximum possible number of assignments. y is found by solving the following optimization problem (with constraints 4.3, 4.4 and 4.5):

$$\underset{x}{\text{maximize}}\Big(\sum_{t=1}^{T}\sum_{c=1}^{C} x_{tc}\Big).$$

Now the minimum number of teacher-course assignments can be added as a constraint (see 4.6).

For the overload (and underload similarly), a constraint like the following is needed:

$$\delta_t^+ = \max\left(\Big(\sum_{c=1}^{C} x_{tc} \cdot L_{tc}\Big) - W_{tc},\ 0\right).$$

For Δ, the following:

$$\Delta = \underset{t\in[1,...,T]}{max} \frac{\delta_t^+ + \delta_t^-}{W_t}.$$

Whilst δ_t^+ and δ_t^- are integer variables, Δ can also take real – and therefore indiscreet – values. The problem is that these are nonlinear functions. This problem can be bypassed by defining Δ, δ_t^+, and δ_t^- as decision parameters and introducing constraints 4.7, 4.8, and 4.9. Constraint 4.7 in combination with constraint 4.9 makes sure, δ_t^+ and δ_t^- are at least as big as the actual over- and underload. Constraint 4.8 sets Δ to be the maximum relative over-/underload, assuming δ_t^+ and δ_t^- represent the actual over- and underload. As Δ is included in the cost function and is to be minimized, this must is fact be the case. This way linearity is maintained. [8]

$$\sum_{t=1}^{T}\sum_{c=1}^{C} x_{tc} \geq y \tag{4.6}$$

$$\delta_t^+ + \delta_t^- = \Big(\sum_{c}^{C} x_{tc} \cdot L_c\Big) - W_t \qquad \forall t \in [1,...,T] \tag{4.7}$$

$$\frac{\delta_t^+ + \delta_t^-}{W_t} \leq \Delta \qquad \forall t \in [1,...,T] \tag{4.8}$$

$$\Delta, \delta_t^+, \delta_t^- \geq 0 \qquad \forall t \in [1,...,T]. \tag{4.9}$$

5 Implementation

5.1 Frontend

The main objective of the the frontend are simplicity and functionality. There is no particular focus on design. For the manual creation of an AP, the spreadsheet software Microsoft Excel [1] has been used. All teachers enter the required data (see Chapter 2) in one shared Excel document. Once an AP is finished, it is sent out as an Excel sheet to the teachers. To maintain this workflow as much as possible, both input and output of the program will be in the form of an Excel sheet. Advantages that come with this is that the data do not have to get entered manually into the program and that no new spreadsheet editor needs to be developed as Excel already offers a nice–looking view on the data. A disadvantage on the other hand is that the Excel file needs to be formatted in a very specific way. Already small changes in the file can require changing many lines of code.

5.1.1 Input

Two Excel files are needed as an input: One is the "availabilities file", containing all the information specific about this assignment plan. It consists of two sheets - one that contains information about the courses and the availabilities / preferences of the teachers (see Figure 5.1) and another which contains information about the desired number of workdays for each teacher.

The other file is the "coordinates sheet", which also consists out of two sheets. They contain the coordinates of all courses and all teachers respectively. Because of this, the program does not require an internet connection but can pull the coordinates from the sheets. The locations usually don't change from one year to another and the sheet can thus stay unmodified.

In order to ensure the right format of both files and all four sheets, pre-filled files exist. They consist of dummy data, which can easily be replaced with the actual data by the user.

5.1.2 Output

The output is also provided in the form of an Excel file. For one it shows which teacher was assigned to which course and which courses are still missing teachers. If no teacher was found for a teaching spot it is marked with an "X". If a teacher was found, their name is written in the according spot with a text color based on the preference they had to teach the course. This gives a good visual overview at first glance. This can be seen

	Courses				Available teachers			
ID	Place	Length	Needed	Overlaps	Pref. 0	Pref. 1	Pref. 2	Pref. 3
0	AT-6020	2,5	2	6	Olivia, Emma	Charlotte	Lukas	
1	AT-6020	2,5	2			Jonas		
2	DE-83645	2,5	2		Joel	Jonas, Olivia		
3	DE-93453	8	3	9	Colt			Griffin
4	DE-93453	3,5	2		Jonas		Lukas, Emma	

Figure 5.1: An example input sheet specifying the availabilities and preferences of the teachers

Course-ID	Assigned teachers			
0	Riggs			
1	Luciana	Meadow	Lane	Morgan
2	XXX			
3	XXX	XXX	XXX	XXX
4	Riggs	XXX		
5	Nicholas	Meadow	Lane	XXX
6	Jaylin	Meadow	Lane	Morgan
7	Jaylin	Meadow	Lane	XXX
8	XXX	XXX		
9	Jaylin	Nicholas	Meadow	Morgan
10	Jaylin	Nicholas	Morgan	
11	Phoebe	Luciana	Meadow	Riggs
12	Dante	Jaylin	Meadow	Lane
13	Nicholas	Lane	XXX	XXX

Figure 5.2: An assignment plan produced by the program

in Figure 5.2. For another it shows relevant data like how many kilometers of euclidean distance need to be covered, how much the number of assigned workdays deviates from the number of desired workdays on average, etc (see Figure 5.3). This way users can analyze the solution, comprehend what made the program choose this solution and get an idea how to change the parameters to get an output that they deem good. The output being an Excel file also offers the option for the user to insert formulas to be able to further analyze the solution.

5.1.3 Graphical User Interface

An additional Graphical User Interface (GUI) was created in order to let the user select the location of the two input files and the location where the output should be placed (see Figure 5.4). Once these three locations are selected, a button appears which can be clicked in order to start the program. Once the calculation is finished, the output Excel file gets placed in the specified location.

The users also have the option to go to the “Custom Weights”-page, as seen in Figure 5.5.

Teacher	Workdays	Desired Workdays	Deviation in %
Dante	14,75	8,25	79%
Aleena	0	10,75	-100%
Phoebe	21	21	0%
Luciana	21,25	9,5	124%
Jaylin	51	21,25	140%
Nicholas	19,5	10,5	86%
Meadow	48	24,25	98%
Lane	44	20	120%
Riggs	6,25	24,25	-74%
Morgan	25,75	26,5	-3%
Average deviation		82%	
Maximum deviation		140%	
Travel distance		73280	
Average preferance		0,96	
Average absolute deviation		13,4	

Figure 5.3: An excerpt of the statistics given by the program about an assignment plan

There they can use the sliders to modify the three weighting parameters mentioned in Section 4.3 for the next calculation.

For creating the GUI the standard Python interface to the Tcl/Tk GUI toolkit Tkinter [2] was used. Alleged advantages of it are that it is simple, stable and easy to use [17]. Yet it turned out to be quite cumbersome even for this task of very small scale.

5.2 Backend

The input sheet with the teachers' availabilities / preferences is transformed into a set of tuples which represent possible assignments. An assignment tuple consists of the name of a teacher, the identification number of the course they could get assigned to, their preference to get assigned to the course and the distance that they have to travel if assigned to the course (see Figure 5.6). For this paper the travel distance is set to twice the euclidean distance between living place of the teacher and the course location. This makes it possible to run the program without an internet connection and independent of any external service. The duration in the tuple is the actual duration multiplied by two. As the duration of courses is specified in half-day steps, multiplying by two ensures it is a whole number and thus an integer variable can be used.

The objective is now to find the combination of tuples which results in the minimal possible cost – calculated with Equation (4.1) – while satisfying all constraints from Section 4.4. Two different solvers are used to achieve this in order to offer an interesting comparison and to back-check if the same optimal results are delivered. Indeed this proved to be helpful many times during the development.

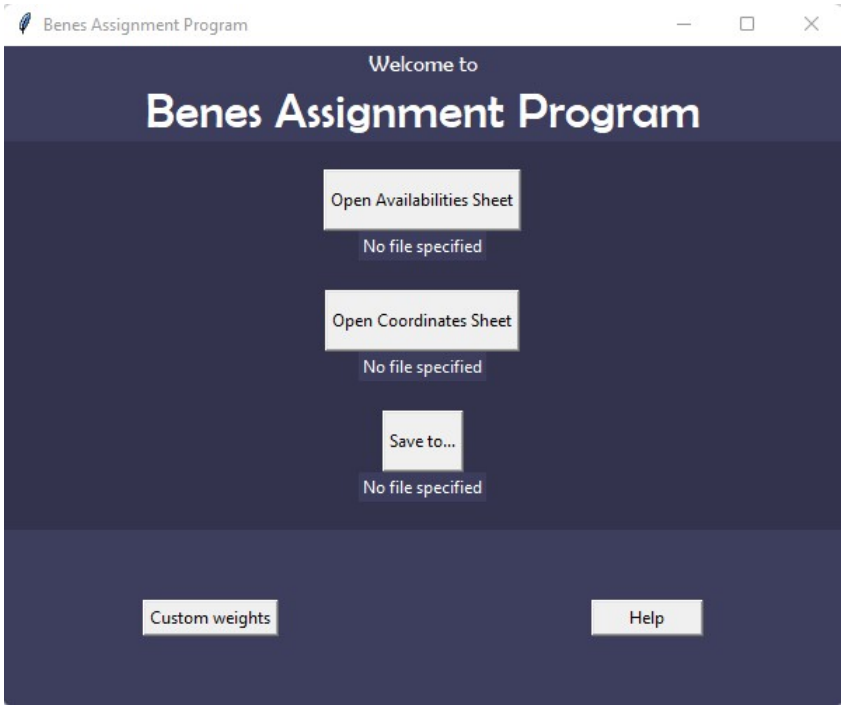


Figure 5.4: The start page of the program

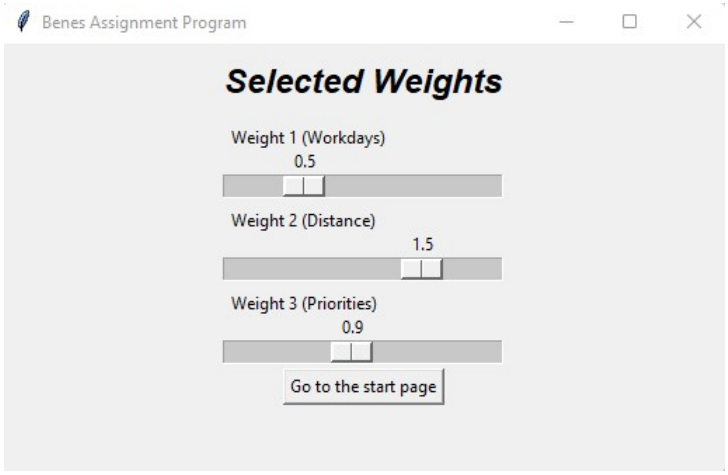


Figure 5.5: The user is offered to set custom weighting parameters

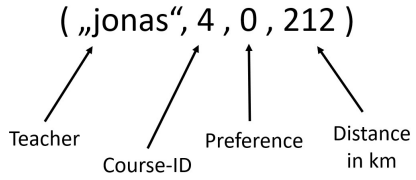


Figure 5.6: The structure of tuples representing (possible) assignments

```
solution = assignment_model.solve(pulp.PULP_CBC_CMD())
```

Figure 5.7: PuLP statement to solve a previously-defined optimization model.

5.2.1 PuLP

PuLP, which is an open-source Python modeling interface, was the first choice to solve the optimization problem at hand. The way PuLP works is that it converts the Python-PuLP expressions into a format that is then exposed to the solver chosen by the user. It supports open-source solvers like GLPK [14] but also commercial solvers like Gurobi [4]. Also it promises to offer a very clean, simple, "pythonic" syntax. [16]

This modular approach makes it very easy for the programmer to change the underlying solver – usually only one line of code needs to be changed. Hence, a programmer can develop using open source software and once it is needed in production there is the possibility to quickly change the free solver to a commercial one, which tend to be much more powerful [11]. The solver used in this paper is the open-source solver CBC [10], as amongst the open-source solvers supported by PuLP it performs very well [15]. The command stating that the previously formalized optimization problem should be solved with the solver CBC can be seen in Figure 5.7.

Another feature of PuLP is that when solving an optimization problem it lets you select a time limit. If the time runs out, the best solution found until this point gets returned. It also lets you select an absolute or a relative gap tolerance regarding the costs which need to be minimized. This can be useful if near-optimal solutions are sufficient and computation time should be reduced.

In fact, PuLP lives up to its promise and offers a clean syntax. Statements are very straightforward and similar to the mathematical formulation. This can be seen when comparing the Python-PuLP statement in Figure 5.8a to the according mathematical formulation of the constraint.

5.2.2 Z3 with the vZ extension

The other approach was to use the SMT Solver Z3, which is freely available from Microsoft Research [7]. SMT stands for "satisfiability modulo theories", which is about determining the satisfiability of one or multiple formulas. As Z3 belongs to this class, it is by itself only able to determine the satisfiability of a formula [3]. However, combined with its

```
for assignment in preference3_assignments:
    assignment_model += (
        x[assignment] == 1,
        f"Must_use_assignment_{assignment}"
    )
```

(a) Using Python-PuLP

```
for assignment in preference3_assignments:
    assignment_model.add(
        x[assignment] == 1,
    )
```

(b) Using Python-vZ

Figure 5.8: Using PuLP and the vZ extension of Z3 to enforce constraint 4.4

extension vZ, one is also able to pose and solve linear optimization problems [5].

Just like PuLP, the vZ extension offered an easy to use syntax. The statement that assignments with preference 3 must be used can be seen in Figure 5.8b. It can be seen that it makes use of similar syntax like PuLP, when comparing it to Figure 5.8a. Yet, in some cases it is less straightforward, for example when building the sum of all assignments.

During the development faulty behaviour of the vZ extension became evident. When adding a certain constraint to a certain optimization problem, the optimum improved. As the solution with the additional constraint turns out to be a viable solution, the solution before adding the constraint must be non-optimal. As a consequence this issue had been reported in the form of a GitHub Issue but as of today it is unanswered [21]. The origin of this bug seems to lie in the mixing of real and integer values in the objective function. It is bypassed by declaring all decision variables as real values.

6 Evaluation

6.1 Computation Time

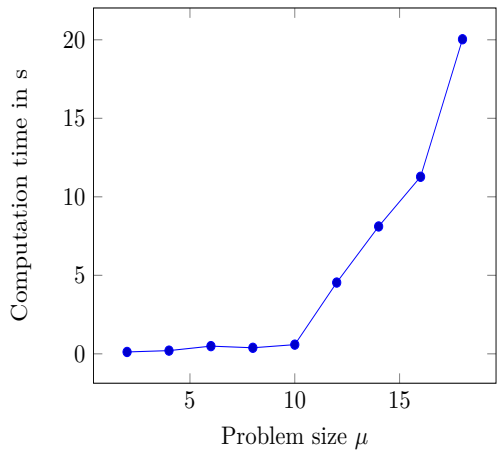
A key element when comparing different solvers is the computation time. Problems like the one of this paper scale very badly, as seen in Section 4.1. The program should deliver a result within a reasonable time, otherwise it will not be used.

A test environment is set up in order to benchmark the computation times for various hypothetical teacher-assignment scenarios. To make sure the program does not just work on few actually tested scenarios, the creation of test-scenarios is automated. The test-scenarios are randomly generated but in a way that each of them resembles a realistic scenario that could occur in the future. To achieve that, the scenarios of the past years have been analyzed. It can be observed, that the scenarios are of different problem sizes, varying in the amount of teachers and courses, yet are similar when it comes to the remaining parameters.

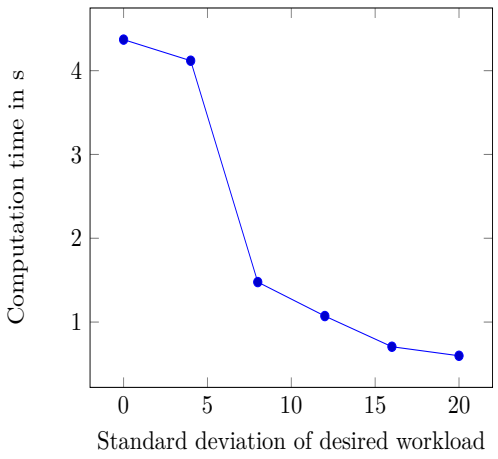
On average, there are twice as many courses as teachers. The chance of two courses overlapping in time is 10%. A course needs on average 2.1 teachers and lasts 5 days. A teacher specifies 21 desired workdays on average, is available for 8 courses and is 350km away from a course location. In consequence, the relationship between overall desired and available workload is relatively balanced. Let μ be the problem size of a scenario. It equals the number of teachers: $\mu = T$. The average problem size of scenarios in the past has been $\mu = 13$. Depending on μ, the remaining parameters could now be calculated. But this does not resemble reality, as usually for example the number of desired workdays varies greatly amongst teachers. To take this into account, a gaussian distribution is used for the different parameters. The standard deviation of these distributions is also averaged over the assignment-scenarios of the past years: 15 for the desired workdays, 0.3 for the number of teachers needed for the courses, 2 for the duration of the courses and 100 for the distance between teachers and courses. The number of courses a teacher is available for is set to 0.4 times the number of workdays he/she desires. This prevents unrealistic scenarios where the two parameters do not correlate.

Particularly eye-catching is the high deviation of desired workdays. It is common for some teachers to only desire 5 workdays, while others desire 70.

The computation time of PuLP with CBC as an underlying solver and of vZ is tested for different problem sizes. As even scenarios with the same problem size can differ greatly, for each of the tested problem sizes, 30 different randomized scenarios are created. The computation of an optimum gets cancelled when the one minute mark is surpassed. It becomes apparent that – for these scenarios – CBC is much more powerful. For problem size $\mu = 4$ CBC takes 0.20 seconds on average, vZ 8.51 seconds. An optimum



(a) Increasing problem size and fixed standard deviation of $\sigma = 15$



(b) Increasing standard deviation of desired workload and fixed problem size $\mu = 10$

Figure 6.1: Computation time to find the optimum using PuLP with the CBC solver

for problem size $\mu = 6$ is found within 0.49 seconds by CBC, vZ takes at least several minutes and had to be cancelled. This makes the vZ extension unsuitable for this problem. Consequently, CBC will be used for the final version of the program. CBC stays almost constant in computation time for higher problem sizes, until it reaches $\sigma = 10$, where it still only needs 0.59 seconds. A a comparison: Just checking satisfiability with Z3 takes 0.13 seconds. For problem sizes $\mu \geq 11$ the computation time grows much more quickly, yet still does not seem to be exponential. This can be seen in the form of a bend in Figure 6.1a. For problems of size $\mu = 18$, 20.0 seconds are needed. Problems of size $\mu = 13$, which is the typical problem size, take around 6 seconds on average. This is acceptable, yet it needs to be mentioned that the execution times vary greatly between scenarios even with the same problem size. A time limit should therefore be set for the final version of the program.

An interesting observation can be made when having a fixed problem size with varying standard deviation of desired workdays (see Figure 6.1b). The more homogeneous the number of desired workdays, the longer the computation takes. This is probably less due to the number of desired workdays and more due to the number of courses teachers are available for, which is strictly coupled to the former.

6.2 Output Quality

The solutions of the program for past scenarios are very similar to the actual APs that had been created manually for those scenarios. This shows that the solutions are sensible. And in fact they are, according to the DM, in some ways better than the solutions obtained by manual creation. Using the program with standard weights, the distance travelled was minimized more, while the rest of the criteria stayed equally good.

An interesting observation can be made when varying the weights. The solution is always acceptable, optimizing all three criteria mentioned in Chapter 2. Just which of them are more optimized slightly changes. However, very different assignments are chosen to achieve this. It becomes clear that there are many different possibilities to achieve similar results, and the program does in fact take them into account. It is therefore easy to produce many different but sensible solutions by just slightly varying the weights. The DM does now not have the job to create a whole assignment plan from scratch anymore, but only to choose the – in his eyes – best one from several good APs. Hence, there is the potential of finding a good solution in a fraction of the time previously needed. If users see how one or several assignments of teachers to courses could improve the overall assignment plan, they can pre-enter those assignments by marking the teachers for these courses with preference 3. This way even criteria which are not part of the criteria mentioned in Section 4.3 but are important to the user can make their way into the assignment plan.

7 Conclusion

The created program allows users to import data describing the specific problem and gives them the possibility to change three weighting parameters. After computation, the solution and its statistics are given. This helps users decide which parameters should be changed, in case they are not satisfied. On top of this, the users are enabled to enforce assignments. The program is therefore not limited to be run just once and deliver the one optimal solution. Instead, it can be seen as a tool for the DM, which allows them to facilitate the process of finding the in their eyes best solution, even if criteria which cannot be represented by the cost function are important to them. This makes the program more versatile.

The computation is done within reasonable time. PuLP with the CBC-solver turns out to be the better choice compared to the vZ extension of Z3, mainly due to shorter computation time. An AP can on average be created within a few seconds. But it can also be seen, that for different problems the computation time varies greatly. A property, which all problems that were analyzed for this paper share is the high deviation between the desired workloads of the teachers. This turns out to reduce computation time significantly. For other problems with different properties, more benchmarks need to be done. If computation time needs to be further reduced, the CBC-solver can be replaced by a more powerful, commercial solver.

Out of many possible solution approaches, the one of using MILP is selected. It guarantees to offer the optimal solution, according to the defined cost function. If new criteria need to be added, only the cost function has to be modified. Still, the computation time is at an acceptable level. The downside is that the standard deviation of the workload needs to be approximated. For the problem at hand, this works well and is admissible. For larger problem sizes the approximation could be less accurate and a different approach could be needed.

Acronyms

AP Assignment Plan

DM Decision Maker

GUI Graphical User Interface

IP Integer Programming

MILP Mixed-Integer Linear Programming

MIP Mixed-Integer Programming

NLO Non-Linear Optimization

TAP Teacher Assignment Problem

Bibliography

[1] Microsoft Excel. `https://www.microsoft.com/en-us/microsoft-365/excel`. Accessed: 2022-08-10.

[2] Tkinter documentation. `https://docs.python.org/3/library/tkinter.html`. Accessed: 2022-08-02.

[3] C. Barrett and C. Tinelli. Satisfiability modulo theories. In *Handbook of model checking*, pages 305–343. Springer, 2018.

[4] B. Bixby. The Gurobi optimizer. *Transp. Re-search Part B*, 41(2):159–178, 2007.

[5] N. Bjørner, A.-D. Phan, and L. Fleckenstein. νZ - an optimizing SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 194–199. Springer, 2015.

[6] B. Chachuat. Mixed-integer linear programming (MILP): Model formulation. *McMaster University Department of Chemical Engineering*, 17, 2019.

[7] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer, 2008.

[8] B. Domenech and A. Lusa. A MILP model for the teacher assignment problem considering teachers' preferences. *European Journal of Operational Research*, 249(3):1153–1160, 2016.

[9] S. Faudzi, S. Abdul-Rahman, and R. Abd Rahman. An assignment problem and its application in education domain: A review and potential path. *Advances in Operations Research*, 2018, 2018.

[10] J. J. Forrest, L. Hafer, J. P. Goncalves, H. G. Santos, and S. S. Brito. Coin-or branch and cut. `https://github.com/coin-or/Cbc`. Accessed: 2022-08-01.

[11] J. L. Gearhart, K. L. Adair, J. D. Durfee, K. A. Jones, N. Martin, and R. J. Detry. Comparison of open-source linear programming solvers. Technical report, Sandia National Lab.(SNL-NM), Albuquerque, NM (United States), 2013.

[12] J. Hartmanis. Computers and intractability: a guide to the theory of NP-completeness (Michael R. Garey and David S. Johnson). *Siam Review*, 24(1):90, 1982.

[13] S. Heipcke. Comparing constraint programming and mathematical programming approaches to discrete optimisation—the change problem. *Journal of the Operational Research Society*, 50(6):581–595, 1999.

[14] A. Makhorin. GLPK (GNU linear programming kit). *http://www.gnu.org/s/glpk/glpk.html*, 2008.

[15] B. Meindl and M. Templ. Analysis of commercial and free and open source solvers for linear optimization problems. *Eurostat and Statistics Netherlands within the project ESSnet on common tools and harmonised methodology for SDC in the ESS*, 20, 2012.

[16] S. Mitchell, M. O'Sullivan, and I. Dunning. PuLP: a linear programming toolkit for Python. *The University of Auckland, Auckland, New Zealand*, 65, 2011.

[17] A. D. Moore. *Python GUI Programming with Tkinter: Develop responsive and powerful GUI applications with Tkinter*. Packt Publishing Ltd, 2018.

[18] D. W. Pentico. Assignment problems: A golden anniversary survey. *European Journal of Operational Research*, 176(2):774–793, 2007.

[19] J. D. Pintér. How difficult is nonlinear optimization? A practical solver tuning approach, with illustrative results. *Annals of Operations Research*, 265(1):119–141, 2018.

[20] M. Rentmeesters, W. Tsai, and K.-J. Lin. A theory of lexicographic multi-criteria optimization. In *Proceedings of ICECCS '96: 2nd IEEE International Conference on Engineering of Complex Computer Systems (held jointly with 6th CSESAW and 4th IEEE RTAW)*, pages 76–79, 1996.

[21] B. Schenk. Github issue: vZ - optimization problems when type casting. `https://github.com/Z3Prover/z3/issues/5875`. Accessed: 2022-08-02.

[22] B. Schenk. Bene's assignment tool. https://doi.org/10.5281/zenodo.7149243, Oct. 2022.

[23] S. Singh, G. Dubey, and R. Shrivastava. A comparative analysis of assignment problem. *IOSR Journal of Engineering*, 2(8):01–15, 2012.

[24] L. A. Wolsey. *Integer programming*. John Wiley & Sons, 2020.